Nginx, HAProxy, ingress và API gateway

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Từ trình duyệt đến tiến trình ứng dụng

Người dùng gõ tên miền. DNS trả địa chỉ; client tạo kết nối TCP; TLS xác thực máy chủ và mã hóa; HTTP request tới load balancer/reverse proxy; proxy định tuyến tới ứng dụng; ứng dụng gọi database/cache/queue rồi trả response. Với Kubernetes, đường đi có thể thêm cloud load balancer -> ingress controller/gateway -> Service -> Pod. Mỗi hop có log, timeout, giới hạn kết nối và health check khác nhau.

502 Bad Gateway thường cho thấy proxy không nhận được phản hồi hợp lệ từ upstream; **503** có thể do thiếu backend sẵn sàng; **504** là timeout ở một gateway. Đây là manh mối, không phải chẩn đoán cuối. Cần xem **proxy nào** trả mã, upstream status, request ID, thời gian, pod endpoints và log ứng dụng. Một request chậm 30 giây có thể bị timeout ở proxy dù ứng dụng vẫn xử lý xong và ghi database; retry từ client khi đó có nguy cơ tạo thao tác trùng.

Bài tập đọc luồng lỗi

Notes API local trả **/health** bình thường nhưng người dùng qua tên miền nhận 502. Liệt kê kiểm tra từ ngoài vào: DNS trỏ đúng chưa; TLS handshake; proxy có route đúng path/host; upstream host/port; service process có lắng nghe; firewall; application log. Với mỗi bước viết lệnh hoặc tín hiệu định dùng và kết quả nào sẽ loại trừ giả thuyết. Không bắt đầu bằng restart mọi thành phần: restart có thể làm mất bằng chứng.

Nginx, HAProxy, web server và PHP-FPM

Nginx có thể phục vụ file tĩnh, reverse proxy, TLS termination và load balancing. HAProxy mạnh ở proxy/load balancing tầng TCP/HTTP, health check và điều tiết traffic. Hai công cụ có vùng giao nhau; lựa chọn phụ thuộc lưu lượng, cách cấu hình, hệ sinh thái và yêu cầu vận hành. Với PHP stack, Nginx nhận HTTP rồi chuyển yêu cầu PHP qua FastCGI tới PHP-FPM; PHP-FPM quản lý worker thực thi PHP. Trả file tĩnh và chạy PHP là hai công việc khác.

Một cấu hình proxy cần các quyết định rõ: upstream nào, thuật toán phân phối, timeout connect/read/write, kích thước request, header **Host**/**X-Forwarded-***, TLS certificate, health check, log format và giới hạn kết nối. Health check quá nông (**200** từ web server) có thể bỏ sót database chết; quá sâu lại có thể làm toàn backend bị loại khi một dependency tạm chậm. Header forwarded sai có thể khiến app tạo URL HTTP thay vì HTTPS hoặc ghi sai IP client. Chỉ tin forwarded headers từ proxy đáng tin; client có thể tự gửi header giả.

Lab 1 - Reverse proxy cho Notes API

Trên máy lab có Docker, chạy Notes API từ **lesson/labs/notes-api/compose.yaml** rồi thêm một Nginx container cùng network. Cấu hình một **server** lắng nghe cổng lab và **proxy_pass** tới tên service/port Notes API; giữ **Host**, gửi request ID nếu cần. Gọi **GET /health** qua proxy, so response trực tiếp qua app. Sau đó cố ý đổi upstream port sai; quan sát HTTP status và error log của Nginx. Sửa lại và xác nhận phục hồi. **Kết quả mong đợi:** đường trực tiếp còn sống khi proxy lỗi, chứng minh phạm vi sự cố ở edge. **Dọn dẹp:** dừng stack và xóa container lab; giữ database volume nếu còn dùng ở module khác.

Tình huống tải cao

Proxy nhận 10.000 kết nối nhưng PHP-FPM chỉ có ít worker, database query chậm. Tăng **max_children** bừa có thể tăng memory và làm DB quá tải. Hãy lấy queue length, worker busy, request latency, DB connection/slow query và memory trước. Nếu nhiều request đang chờ cùng một dependency, tăng worker chỉ chuyển hàng đợi xuống DB. Đây là ví dụ cần quan sát toàn đường đi.

Ingress controller, API gateway và identity provider

Ingress là tài nguyên mô tả quy tắc HTTP routing trong Kubernetes; cần ingress controller chạy để áp dụng. Nginx ingress controller là một cách thực hiện. Kubernetes Gateway API là mô hình resource khác với vai trò tách người quản trị hạ tầng và route ứng dụng. API gateway như Kong hoặc WSO2 có thể thêm auth, rate limit, transformation, developer portal hoặc quản trị API. Đừng thêm gateway khi chỉ cần route đơn giản: mỗi lớp thêm độ phức tạp, latency và điểm lỗi.

Keycloak là identity and access management: xác thực người dùng, cấp token theo giao thức như OAuth 2.0/OpenID Connect và quản lý realm/client/role. API gateway có thể kiểm tra token trước khi chuyển request, nhưng ứng dụng vẫn phải kiểm tra quyền phù hợp nghiệp vụ. TLS bảo vệ đường truyền; token chứng minh danh tính/ủy quyền theo điều kiện; chúng không thay thế nhau. Một token có chữ ký hợp lệ nhưng sai audience hoặc hết hạn vẫn phải bị từ chối. Cần phân biệt lỗi 401 (chưa xác thực) với 403 (đã xác thực nhưng thiếu quyền) trong chính sách của app/gateway.

Bài tập thiết kế gateway

Ba microservice **/users**, **/orders**, **/admin**. Dịch vụ admin chỉ cho một nhóm nội bộ. Vẽ nơi kết thúc TLS, nơi kiểm tra token, nơi áp rate limit, đường log và request ID. Nếu gateway hỏng, có bypass trực tiếp tới service không? Nếu có, chính sách auth ở gateway có thể bị vượt qua; hãy nói cách giới hạn network và/hoặc kiểm tra token ở service. Với Keycloak, mô tả quy trình rotation key/cert và cách dịch vụ lấy JWKS khi mạng gặp lỗi.

Đọc một lỗi 502/503/504 theo lớp

Người dùng gửi HTTPS tới LB/reverse proxy; proxy chọn upstream qua DNS hoặc service discovery; upstream có thể là Nginx/PHP-FPM hoặc Kubernetes Service/Pod. **502** là phản hồi không hợp lệ từ upstream trong nhiều proxy; **503** thường do backend không sẵn sàng hoặc từ chối; **504** gợi ý timeout. Nhưng sản phẩm/config có thể khác nhau: phải xác định **thành phần nào tạo status**, xem access/error log và upstream status/time. Một request được proxy trả 504 vẫn có thể tiếp tục chạy ở app; client retry có thể tạo giao dịch trùng.

Lab Nginx cho Notes API

Tại **lesson/labs/edge-local/**, chạy **docker compose up --build -d**. Nginx lắng nghe cổng 8080, **proxy_pass http://notes-api:8000**, chuyển **Host** và request ID. Gọi **GET /health** qua 8080 và trực tiếp 8000; ghi status/latency. Đổi upstream trong **nginx.conf** thành **notes-api:9999**, tái tạo Nginx container, gọi lại và đọc error log để xác nhận **connect() failed** hoặc lỗi tương ứng. Sửa về 8000; kiểm tra phục hồi. Dọn bằng **docker compose down** và chỉ thêm **-v** nếu chắc chắn không cần dữ liệu lab. Nếu chưa cài Docker, đọc config và giải thích vì sao hostname service được phân giải trong network Compose, còn **localhost** trong Nginx container không trỏ về Notes API.

Load balancing và health check

Round robin, least connections hoặc weight giải vấn đề phân phối khác nhau; một backend chậm có thể giữ nhiều kết nối. Health check phải đủ phát hiện backend không phục vụ request quan trọng nhưng không phụ thuộc quá sâu khiến mọi backend bị loại khi một dependency chung lỗi. Cần timeout, retry, circuit breaking và connection limits có chủ đích. Nếu ba backend cùng gọi một DB đang chết, chuyển traffic qua lại không khắc phục DB. Khi thiết kế HA, xét LB cũng cần HA và DNS/route trỏ đúng khi một LB mất.

Ingress, gateway, PHP-FPM và identity

Ingress controller áp route HTTP trong Kubernetes; API gateway có thể thêm auth, quota, transform, chính sách API; Keycloak cung cấp identity/token; PHP-FPM chạy code PHP phía sau web server. Chúng có thể cùng có mặt nhưng trách nhiệm khác nhau. Nếu auth chỉ ở gateway, kiểm tra service không bị truy cập bypass và token đúng issuer/audience/expiry. Nếu Nginx nói chuyện với PHP-FPM, kiểm tra pool worker, queue và memory khi tải cao; tăng worker bừa có thể làm DB quá tải. Luôn giữ request ID qua các hop để nối log.

Tình huống phỏng vấn

Pod Ready, Service endpoints có, ingress trả 503 cho một path. Kiểm tra rule host/path, ingress controller event/log, service port/targetPort, NetworkPolicy và app route. Nếu chỉ một path lỗi, có thể rule rewrite hoặc app route thay đổi; không cần kết luận cluster hỏng. Trả lời theo bằng chứng: kết quả **curl** trực tiếp Service/Pod so với qua ingress, rồi diff cấu hình release gần nhất.

Edge là nơi nhiều lớp cùng có thể trả lỗi

Một request từ Internet có thể đi qua DNS, CDN/WAF, external LB, Nginx/HAProxy, Kubernetes ingress controller, Service, Pod, PHP-FPM/app và DB. Khi thấy 502, 503 hoặc 504, trước tiên xác định **lớp nào phát mã** từ response headers và log tương ứng. 502 thường gợi proxy nhận response upstream không hợp lệ hoặc không kết nối được; 503 có thể là không có backend/overload/maintenance; 504 là timeout ở một proxy. Ý nghĩa chi tiết phụ thuộc sản phẩm/config. Một request có thể nhận 502 từ Nginx dù app hoàn toàn không nhận request; sửa app code khi chưa kiểm tra upstream sẽ mất thời gian.

Reverse proxy nhận request rồi gửi tới upstream. Load balancer chọn một trong nhiều backend theo thuật toán/health; web server có thể phục vụ static và proxy dynamic; PHP-FPM thực thi PHP qua FastCGI, không phải HTTP server công khai theo cách Nginx là. API gateway có thêm auth, rate limit,

routing/transform/policy; identity provider như Keycloak phát hành/quản lý danh tính/tokens theo giao thức, không thay app authorization từng object. Kong/WSO2 có tập tính năng riêng và overhead vận hành; chọn sau khi rõ nhu cầu, không thêm gateway chỉ vì có trong JD.

Port, Host, path và TLS qua ví dụ

Client gọi **https://api.example.com/v1/notes**. TLS có thể kết thúc ở LB; Nginx nhận HTTP với Host **api.example.com**, match **/v1/**, proxy tới app **notes:8000**. Nếu Nginx gửi nguyên **/v1/notes** nhưng app chỉ có route **/notes**, app trả 404; nếu Nginx strip prefix đúng, request thành công. Dấu **/** ở **proxy_pass** và rule rewrite có thể thay path theo cú pháp sản phẩm; không suy đoán, đọc config render và thử request cụ thể. Nếu proxy gửi sai **Host** hoặc **X-Forwarded-Proto**, app có thể tạo redirect sai vòng lặp hoặc link HTTP trong trang HTTPS. Chỉ tin forwarded headers từ proxy đáng tin cậy để tránh spoofing.

TLS ở edge bảo vệ đoạn client->edge; đoạn edge->upstream có thể dùng HTTP hoặc TLS khác. Với môi trường yêu cầu mã hóa end-to-end, xác định chứng chỉ/CA ở từng đoạn, SNI, hostname validation và rotation. Chứng chỉ hết hạn có thể làm client fail trước khi request tới app. Dùng **curl -v**/công cụ certificate để xem thời hạn/chain; không dùng **-k** làm cách chữa production vì bỏ kiểm tra identity server.

Lab Nginx local từ cấu hình có sẵn

Điều kiện: Docker/Compose trên máy lab, file **lesson/labs/edge-local/compose.yaml** và **nginx.conf** đã có. Đọc trước để vẽ đường **client -> Nginx -> upstream**; ghi host port, Nginx listen port, upstream service/port, health path. Chạy Compose theo tài liệu dự án trong đúng thư mục, kiểm tra **docker compose ps**, gọi **curl -i** tới host port và đọc access/error logs. **Kết quả dự kiến:** request qua proxy tới app theo config; tên container và port phải lấy từ file thực, không dùng output giả định.

Gây lỗi trong **bản sao lab**: đổi upstream port sang port không có listener rồi reload/recreate Nginx; request có thể trả 502/504 tùy điều kiện. Đọc error log để thấy **connect() failed** hay timeout, thử gọi upstream trực tiếp từ mạng Compose, sửa port và xác nhận request phục hồi. Lỗi thứ hai: đổi path route sai để nhận 404 từ proxy/app; phân biệt bằng access log của app có request hay không. Dọn đúng Compose project và volume lab, không chạy **docker system prune** trên máy dùng chung.

Ingress trên Kubernetes và Gateway API

Ingress resource mô tả host/path -> Service, nhưng phải có ingress controller triển khai; chỉ **kubectl apply** Ingress không tạo một proxy hoạt động. Kiểm tra IngressClass, controller pods, events, LB address, Service backend và EndpointSlice. Kubernetes docs hiện nói Ingress API đã frozen và khuyến nghị Gateway API cho khả năng mới; Ingress vẫn ổn định và được dùng rộng, không có nghĩa phải migrate ngay. Gateway API tách các vai trò/route theo tài nguyên và implementation cụ thể; đọc khả năng controller đang dùng. Nguồn: Kubernetes Ingress và Gateway API.

Health check và thuật toán không thay capacity

LB health check chỉ tốt bằng endpoint kiểm tra. Nếu chỉ kiểm tra TCP connect, app có thể accept socket nhưng trả 500 mọi request. Nếu health check gọi DB quá nặng và DB chập chờn, toàn bộ backend có thể bị loại cùng lúc. Thiết kế endpoint rẻ, có ý nghĩa "có thể nhận request" và đo qua luồng người dùng riêng. Round robin chia đều số request nhưng request dài/ngắn khác nhau; least connections hoặc weight có thể hữu ích tùy workload/implementation. Không chọn thuật toán trước khi đo bottleneck và capacity downstream.

Ca có lời giải: sau deploy, Nginx 502 cho mọi request; Pod app Ready. Đọc upstream trong config và log Nginx, kiểm tra DNS tên service từ proxy, Service selector/EndpointSlice, port/targetPort và process bind. Pod Ready không chứng minh proxy đang gọi đúng port. Nếu một số request lỗi, xem backend cụ thể, keepalive/timeout, canary revision và app logs theo request ID. **Câu hỏi:** Keycloak đã xác thực user A, API có cho đọc note của user B không? Không tự động; app phải kiểm tra authorization theo object/tenant.

Nguồn: NGINX reverse proxy/load balancing, HAProxy documentation, Keycloak docs.

PHP-FPM và reverse proxy: request chạy ở đâu?

Với PHP stack, Nginx nhận HTTP/TLS, phục vụ static hoặc chuyển PHP request tới PHP-FPM qua FastCGI socket/TCP; PHP code chạy trong worker PHP-FPM và gọi DB. Nếu Nginx trả 502, có thể socket PHP-FPM không tồn tại, sai quyền socket, pool worker hết/timeout hoặc PHP-FPM chết. Nếu Nginx phục vụ CSS được nhưng **/index.php** lỗi, DNS/TLS/edge tới Nginx đã đi qua; tập trung FastCGI/PHP-FPM. Nếu PHP-FPM log báo **max_children reached**, tăng worker có thể giúp nhưng phải tính RAM mỗi worker và DB connections; tăng mù có thể làm máy/DB quá tải.

Health check và draining khi release

Một LB bỏ backend sau một lần health check fail có thể tạo dao động nếu endpoint chập chờn; cần threshold/interval và hiểu thời gian phát hiện. Khi rolling deploy, backend cũ cần ngừng nhận request mới, hoàn tất request đang chạy trong grace period, rồi thoát. Nếu proxy còn giữ keepalive tới Pod sắp chết, người dùng có thể thấy 502 rải rác. Đo số request lỗi theo backend/revision và mốc shutdown; chỉnh readiness, preStop/grace period hoặc LB draining theo implementation. Không tăng timeout vô hạn để giấu ứng dụng không shutdown đúng.

Rate limiting, auth và client IP

Rate limit ở edge bảo vệ downstream nhưng có thể chặn người dùng chung NAT nếu key chỉ là IP; nếu key là user ID, cần xác thực trước và chống spoofing. Header **X-Forwarded-For** có thể chứa dữ liệu do client tự gửi nếu proxy không chuẩn hóa; chỉ tin danh sách proxy đã định. API gateway có thể validate token, nhưng app vẫn cần kiểm tra quyền đối tượng (user A không đọc dữ liệu user B). Keycloak/OIDC cung cấp identity/token; token cần kiểm tra issuer, audience, thời hạn và signature theo library/chính sách phù hợp, không chỉ decode base64.

Case có lời giải: chỉ 504 ở endpoint export

GET /export mất 90 giây; proxy timeout 60 giây, app vẫn xử lý và tạo file. Tăng proxy timeout lên 120 giây có thể chữa triệu chứng ngắn hạn nhưng giữ connection/resource rất lâu. Thiết kế tốt hơn có thể là job bất đồng bộ: POST tạo export job với idempotency, trả job ID; worker tạo file, người dùng poll/download khi xong; alert backlog/age. Nếu buộc đồng bộ, đo tải, giới hạn concurrency, deadline và cancellation. Mã 504 từ proxy không tự nói app có hoàn tất hay chưa, nên retry có thể tạo job trùng.

Câu hỏi: Ingress YAML có host/path đúng nhưng không có traffic, vì sao? Kiểm tra ingress controller/IngressClass, LB address/DNS, Service backend/EndpointSlice, TLS, firewall. **502 chỉ từ một backend?** Gắn log/metric theo upstream, kiểm tra instance đó khác config/version/node. Nguồn: Kubernetes Ingress, NGINX admin guide.

Bài tập thứ tự kiểm tra: người dùng báo 503 từ domain public. Bắt đầu bằng request **curl -v** kèm Host/domain thật để biết IP, TLS và response headers. Nếu LB trả 503 trước ingress, xem LB target health; nếu ingress controller trả 503, xem Ingress route, Service và EndpointSlice; nếu app trả 503, xem log/DB/dependency. Ghi một dấu hiệu giúp phân biệt nguồn phát mã ở từng nhánh. Chỉ sau khi có nguồn phát mã mới quyết định sửa LB, ingress hoặc app. Một port-forward vào Pod thành công không chứng minh domain public được route đúng.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.nginx.com/nginx/admin-guide/
- https://www.haproxy.org/documentation/
- https://kubernetes.io/docs/concepts/services-networking/ingress/
- https://www.keycloak.org/documentation

Phụ lục: mã lab để đọc offline

lesson/labs/edge-local/compose.yaml

```
services:
  notes-api:
    build: ../notes-api
    environment:
      NOTES_DB: /data/notes.db
      NOTES_HOST: 0.0.0.0
    volumes:
      - notes-data:/data
    ports:
      - "127.0.0.1:8000:8000"
  nginx:
    image: nginx:1.27-alpine
    depends_on:
      - notes-api
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    ports:
      - "127.0.0.1:8080:8080"
volumes:
  notes-data:
```

lesson/labs/edge-local/nginx.conf

```
server {
    listen 8080;
    server_name localhost;

    location / {
        proxy_pass http://notes-api:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Request-ID $request_id;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 3s;
        proxy_read_timeout 10s;
    }
}
```